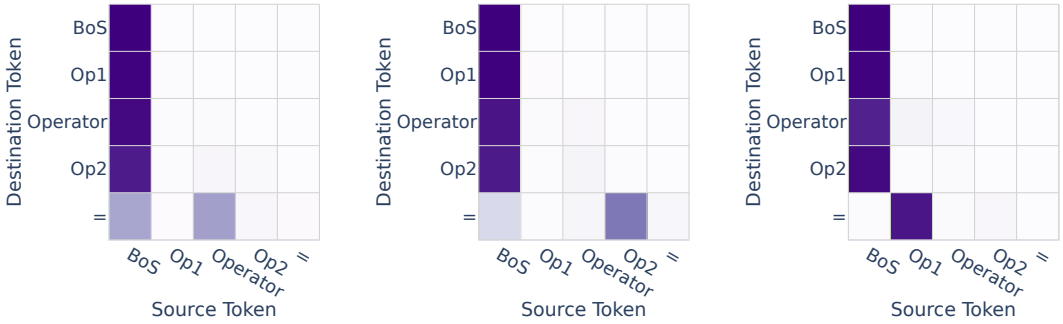
Table 2: Llama3-8B operator-specific arithmetic circuit attention heads. The general arithmetic heads are marked **bold**. LiHj denotes the jth attention head in Layer i.

Operator	Circuit Heads
$+$	**L2H2**, L5H3, L5H31, L14H12, **L15H13**, **L16H21**
$-$	**L2H2**, L13H21, L13H22, L14H12, **L15H13**, **L16H21**
$\times$	**L2H2**, L5H30, L8H15, L9H26, L13H18, L13H21, L13H22, L14H12, L14H13, L15H8, **L15H13**, L15H14, L15H15, L16H3, **L16H21**, L17H24, L17H26, L18H16, L20H2, L22H1
$\div$	**L2H2**, L5H31, **L15H13**, L15H14, **L16H21**, L18H16

to its attended position (e.g., for L15H13 we zero out all patterns that move information to the op_2 position), preventing any influence from previous positions. We observe this does not affect the circuit's performance, indicating that the representation copied by each general arithmetic head to the final position contains information only regarding the token at its original position.



(a) Layer 2 Head 2 (b) Layer 15 Head 13 (c) Layer 16 Head 21

Figure 14: The attention patterns for the general arithmetic attention heads in Llama3-8B, show that each head attends, at the last token, to a single previous token across all prompts. When combined, these heads attend to all three operand and operator positions.

C Multi-layer perceptron implementation details

Across the models we analyze, different implementations exist for the MLP layer in a transformer block. More specifically, Pythia-6.9B and GPT-J use simple MLP layers, consisting of two matrix multiplications. For a layer l, it can be described as:

$$\mathbf{h}^l_{post} = \sigma\left(\mathbf{h}^l_{in}\mathbf{W}^{l^\top}_{in}\right) \tag{4}$$

$$\mathbf{h}^l_{out} = \mathbf{h}^l_{post}\mathbf{W}^l_{out} \tag{5}$$

where $\mathbf{h}^l_{in}, \mathbf{h}^l_{out} \in \mathbb{R}^d$ are the input and output representations of the MLP at layer l, respectively. $\mathbf{h}^l_{post} \in \mathbb{R}^{d_{mlp}}$ is the post-activation vector, $\mathbf{W}^l_{in}, \mathbf{W}^l_{out} \in \mathbb{R}^{d_{mlp}\times d}$ are parameter matrices, and σ is a non-linearity function. Llama3-8B and Llama3-70B use a Gated MLP layer (Liu et al., 2021), described in the following two equations for layer l:

$$\mathbf{h}^l_{post} = \sigma(\mathbf{h}^l_{in}\mathbf{W}^{l^\top}_{gate}) \circ (\mathbf{h}^l_{in}\mathbf{W}^{l^\top}_{in}) \tag{6}$$

$$\mathbf{h}^l_{out} = \mathbf{h}^l_{post}\mathbf{W}^l_{out} \tag{7}$$

where $\mathbf{W}^l_{gate} \in \mathbb{R}^{d_{mlp}\times d}$ is an additional parameter matrix and $\circ$ is Hadamard product. Biases are omitted in both presentations.

While the key-value view that was used in Section 3 was devised for simple MLPs (Geva et al., 2021), it can be applied to the Gated MLP mechanism of Llama3 models as well. For the nth neuron, we treat the element $\mathbf{h}^{l,n}_{post}$ of $\mathbf{h}^{l}_{post}$ as the activation of the nth key vector. Each such activation is multiplied with $\mathbf{v}^{l,n}_{out}$, a row vector of $\mathbf{W}^{l}_{out}$, resulting in the same multiplication as performed in simple MLPs.

D NEURON INTERSECTION BETWEEN OPERATORS

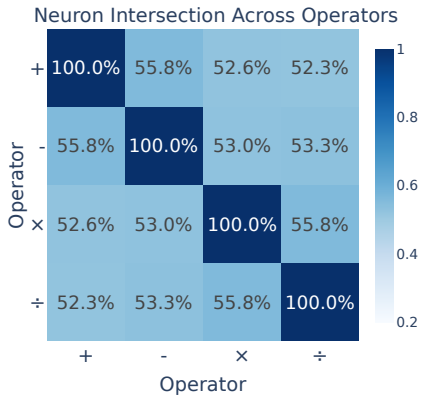


Figure 15: IoU of MLP neurons between operator-specific circuits

Figure 15 presents the IoU (intersection over union) of causally important neuron sets in the arithmetic circuits, identified separately for each operator. Each circuit consists of the top-200 neurons from each layer $l \in [16, 32]$, ranked based on their mean effect (Section 3.1), totaling $3,200$ neurons. The average IoU between any two operator-specific circuits is relatively low (54%), indicating that a substantial proportion of key neurons are unique to each operator. Consequently, we define a distinct circuit for each arithmetic operator at the neuron level (Section 3.1).

E HEURISTIC TYPES DESCRIPTIONS

In this section, we present the various heuristic types that were manually identified.

Most heuristic types are defined with parameters. A neuron is defined as a heuristic H, if it matches the heuristic definition given a parameter value. For example, the neuron in Figure 1b is a range heuristic with a range parameter $[150, 180]$. In Algorithm 1, we describe the process of matching between each important neuron and each heuristic H. The parameters are sub-sampled evenly to cover most observed possibilities, which causes some inaccuracies (for example, a neuron that fires when $op_2 \in [105, 205]$ will be classified as a range heuristic with $op_2 \in [100, 200]$ due to our parameter choices).

Several types of heuristics (Range, Modulo, Pattern) can apply either to an operand in a prompt or to the prompt's result. For example, an "operand range" heuristic is triggered when op_1 or op_2 falls within a specific numerical range, while a "result range" heuristic activates when the prompt's ground truth result is within a defined range.

The full list of heuristic types is as follows:

- **Range heuristic:** A neuron that activates when a value (either an operand or result) falls within a specified range $[a, b]$. The parameters $a, b \in \mathbb{N}$ are chosen such that the range length $len = (b - a) \in [10, 30, 50, 100]$ for addition, subtraction, and multiplication, and $len = (b - a) \in [2, 10, 100]$ for division (due to a different distribution of potential results for integer division). The range start, a, is defined as $a \in x_n : x_n = x_0 + n \cdot \max(\lfloor \frac{\text{len}}{3} \rfloor, 10), n \in \mathbb{N}, x_0 = 0$, provided a remains below the maximum prompt result. This definition of range start and length allows for ranges to intersect or overlap.

- **Modulo heuristic:** A neuron that activates when a value (either operand or result) is congruent to m modulo n. The parameters are $n \in \{2, 3, 4, 5, 6, 7, 8, 9, 11, 13, 15\}$ and $m \in [0, n-1]$. We exclude $n = 10$ or $n = 100$ as these are specific cases of the broader "pattern" heuristics, discussed next. Additionally, $n = 12, 14$, and $n \geq 16$ are excluded, as no neurons were classified under these parameter values.
- **Pattern heuristic:** A neuron that activates when a value (either operand or result) matches a specific regular expression p. The parameter p is a 3-digit regular expression, potentially padded with leading zeros. For example, a neuron classified under the "operand pattern 1.2" heuristic activates when one of the operands has 1 in the hundreds place and 2 in the units place.
- **Identical operands heuristic:** A neuron that activates when both operands are equal (op_1 == op_2). The tokens embedded in the value vector change depending on the operator for which the neuron activates. For instance, in subtraction, such neurons have been observed to promote the "0" token (the result of subtracting a number from itself).
- **Multi-result heuristic:** A neuron that promotes a set $\mathbf{S}$ of several unrelated results (For example, a neuron that promote the results $[4, 5, 7]$). This heuristic type is defined exclusively for division, where the parameter S can consist of several values, where $|\mathbf{S}| \in [2, 4]$. This range was chosen based on observations of activation patterns. The values included in $\mathbf{S}$ are chosen according to the result tokens that the neuron contributes the most to: we determine $\mathbf{S}$ by examining the top prompts for which the neuron promotes their answer. From these top answers, we identify a minimal set of 2 to 4 distinct result values. This set of result values is chosen if it accounts for more than a threshold percentage of the results. This approach captures the most significant result values that the neuron consistently promotes in division operations. If no such set of different results exists, a neuron is not classified as this heuristic.

F Heuristic Neuron Classification Algorithm

We present the full algorithm used to match between each neuron and each heuristic type in Algorithm 1 (as exemplified in Figure 6). The goal of this process is to check if a neuron implements a specific heuristic. We repeat this process for each pair of neuron n in layer l and heuristic H. This method allows a single neuron to be classified as several heuristics. The matching of each neuron to heuristics is done separately for each arithmetic operator.

Algorithm 1 Neuron Classification To Heuristic Type

Inputs: Heuristic H, Layer $l \in [1, l_{max}]$, Neuron Index $n \in [1, n_{max}]$,
Operator $o \in \{+, -, \times, \div\}$, Threshold $t \in [0, 1.0]$

1: $\mathbf{A} \leftarrow \text{GenerateActivationPattern}(l, n, o)$ ▷ 2D Activation pattern
2: **if** H is a direct heuristic **then**
3: $\quad \mathbf{l} \leftarrow \text{LogitLens}(\mathbf{v}_{out}^{l,n})$ ▷ Logits over vocabulary
4: $\quad \mathbf{L} \leftarrow \text{ConvertToPattern}(\mathbf{l})$ ▷ 2D Logits Pattern
5: $\quad \mathbf{A} \leftarrow \mathbf{A} \circ \mathbf{L}$ ▷ Element-wise multiplication
6: **end if**
7: $prompts_{heuristic} \leftarrow \text{GetAssociatedPrompts}(H)$ ▷ Expected prompts to activate in H
8: $k \leftarrow |prompts_{heuristic}|$
9: $prompts_{neuron} \leftarrow \text{GetTopKActivatingPrompts}(\mathbf{A}, k)$ ▷ Prompts that activate neuron
10: $prompts_{intersection} \leftarrow prompts_{heuristic} \cap prompts_{neuron}$
11: $score \leftarrow \dfrac{|prompts_{intersection}|}{k}$ ▷ Normalize intersection to $[0, 1.0]$
12: **return** $score \geq t$

The algorithm first calculates the neuron's activations $h_{post}^{l,n}$ for all prompts, yielding a 2D activation pattern (Line 1), as those seen in Appendix J. When checking if a neuron implements a *direct* heuristic (heuristics that directly promote relevant result tokens (Section 3.2)), we also need to take into consideration the tokens that are embedded in the neuron's value vector $\mathbf{v}_{out}^{l,n}$. This is not done in indirect heuristics because we do not expect relevant result tokens to be promoted in such heuristics. Thus, we extract the logits of all numerical tokens ("0", "1", "2", ... "999") from the value vector